

补充模拟卷 02：语义分析、AST、符号表补缺

目录

A 卷题目	2
B 简明答案	3

A 卷题目

一、依赖图与拓扑求值顺序（20 分）

考虑语义规则：

```
E -> E1 + T
    E.val = E1.val + T.val
```

```
E -> T
    E.val = T.val
```

```
T -> num
    T.val = num.lexeme
```

对输入：

```
3 + 4
```

1. 写出主要属性依赖关系。
2. 写出一种合法的属性求值顺序。
3. 计算最终 E.val。

二、受控副作用（20 分）

考虑声明翻译：

```
D -> T id
T -> int
```

语义动作：

```
addType(id.name, T.type)
```

1. 说明该语义动作的副作用是什么。
2. 为什么该动作必须在 T.type 已经确定之后执行。
3. 写出受控副作用应满足的基本要求。

三、CST 与 AST (20 分)

表达式:

$a + b * c$

1. 说明 CST 和 AST 的区别。
2. 画出该表达式的 AST。
3. 说明为什么后续语义分析更常使用 AST。

四、符号表与作用域 (20 分)

考虑程序片段:

```
{  
    int x;  
    {  
        float x;  
    }  
}
```

1. 说明进入和退出代码块时符号表栈如何变化。
2. 在内层代码块中查找 x 时应得到哪个声明。
3. 退出内层代码块后查找 x 时应得到哪个声明。

五、类型信息与标识符绑定 (20 分)

回答:

1. 符号表通常记录哪些信息。
2. AST 中的标识符结点为什么要绑定符号表条目。
3. 符号表对类型检查有什么作用。

B 简明答案

一、依赖图与拓扑求值顺序答案

对 $3 + 4$, 主要属性依赖:

T1.val 依赖 num(3).lexeme
E1.val 依赖 T1.val
T2.val 依赖 num(4).lexeme
E.val 依赖 E1.val 和 T2.val

一种合法求值顺序:

1. T1.val = 3
2. E1.val = T1.val = 3
3. T2.val = 4
4. E.val = E1.val + T2.val = 3 + 4 = 7

最终:

E.val = 7

二、受控副作用答案

副作用:

addType(id.name, T.type) 会修改符号表。

必须在 T.type 确定后执行, 因为符号表中需要记录标识符的正确类型。如果 T.type 尚未计算, 就无法把正确类型写入符号表。

受控副作用要求:

1. 执行位置明确。
2. 执行顺序确定。
3. 不破坏属性依赖关系。
4. 不引入依赖环。
5. 不导致重复插入或错误类型绑定。

三、CST 与 AST 答案

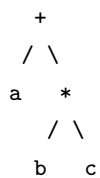
CST:

具体语法树, 完整反映文法推导过程, 包含括号、辅助非终结符等语法细节。

AST:

抽象语法树, 去掉冗余语法细节, 保留表达式和语句的核心结构。

a + b * c 的 AST:



后续语义分析更常使用 AST，因为 AST 更直接表示程序语义结构，便于类型检查、符号绑定、优化和代码生成。

四、符号表与作用域答案

进入外层代码块：

```
push 外层符号表
插入 x : int
```

进入内层代码块：

```
push 内层符号表
插入 x : float
```

内层代码块中查找 x：

```
先查内层符号表，得到 x : float
```

退出内层代码块：

```
pop 内层符号表
```

此后查找 x：

```
得到外层的 x : int
```

退出外层代码块：

```
pop 外层符号表
```

五、类型信息与标识符绑定答案

符号表通常记录：

- 名字
- 类型
- 作用域
- 存储位置
- 参数信息
- 函数信息

AST 中标识符结绑定符号表条目的原因：

同名标识符可能出现在不同作用域中。
绑定后可以明确该标识符对应哪一个声明。

符号表对类型检查的作用：

通过符号表查出变量、函数、表达式的类型，
判断赋值、运算、函数调用等是否类型匹配。